

Complete Software Development Life Cycle (SDLC)

Alexis Y. López Arroyo

Universidad Interamericana de Puerto Rico

COMP3400: Software Engineering

Prof. Edwin Anaya Almeida

12 de mayo de 2026

Index

1. Company description.....	4
2. Problem Identification.....	4
3. System Justification.....	4
4. Project Scope.....	5
5. Stakeholder Identification.....	6
6. Risk Analysis.....	7-8
7. Functional Requirements.....	9
8. Non-Functional Requirements.....	10
9. User Stories.....	11-15
10. Use Case diagram.....	16-18
11. 3 fully developed use cases.....	19-21
12. Traceability Matrix.....	22
13. Preliminary Architectural Proposal.....	23-24
14. Definitive architectural design.....	25
15. Class diagram.....	26
16. Sequence diagram.....	27-29
17. Entity-relationship model.....	30-31
18. Mockups or interface prototypes.....	32
19. Test plan.....	33
20. Test cases.....	34
21. Evidence of test execution.....	35-38
22. Log of errors found.....	39

23. Deployment strategy.....	40
24. Maintenance plan.....	41-43
25. Student's professional reflection.....	44
26. Reference.....	45

Company description

GameStop is a company that primarily sells video games and consoles, including PlayStation, Nintendo, and Xbox. In addition to these products, the store sometimes offers console accessories and gift cards. GameStop also allows trading, where customers can exchange items for store credit to redeem for any products they desire.

Problem identification

GameStop is currently facing challenges related to the swift depreciation of its inventory and the complexities of local regulations concerning secondhand goods. Store employees frequently struggle to assess the physical condition of consoles or disks consistently, which can lead to financial losses when they inadvertently overpay for damaged items. GameStop is currently facing challenges due to the rapid depreciation of its inventory and the complexities of local regulations regarding secondhand goods. Store employees often struggle to consistently assess the physical condition of consoles and discs, which can lead to financial losses. When they inadvertently overpay for damaged items, it results in a profit margin reduction of 15% to 20% per defective unit. This situation highlights the lack of a standardized system within GameStop, as the manual appraisal process takes between 15 and 20 minutes, thereby increasing operating costs. 15% to 20% in profit margin per unit of defective items. which can demonstrate the lack of a standardized system in the Gamestop system, meaning the manual appraisal process takes a long amount of time, between 15 and 20 minutes, increasing operating costs.

System Justification

The system is required to incorporate an automated valuation feature designed to significantly reduce the potential for human error. This feature will not only ensure adherence to legal compliance standards but also maximize profit margins. It will achieve this by automatically calculating various costs associated with reconditioning and storage, thereby providing accurate and efficient financial assessments.

Project Scope

The system must display the available consoles and products, allowing users to pre-order or purchase any item directly through the website. During its development, the system must adhere to local laws and verify customer credentials to minimize human error. This includes validating the customer's age, providing options to remove credit or debit cards from the system, and facilitating multiple shipping options for the desired products.

What will include	What will not include
• The system will restrict access to “M” rated games and automatically remove them from the cart if the user does not meet age requirements. • The platform will display video games tailored to the user's birthdate and ensure game stock is updated frequently. • A dedicated tab for seasonal events, such as Christmas or Black Friday, will be integrated. • “Add to cart/remove from cart” functionality for users. • “Add, change, or remove credit/debit card” functionality.	• The system will not display a private owner's full credit or debit card number. It should be displayed like (***** 1234) • Stock status (Available vs out-of-stock) is displayed only for clients. • Manual Age overrides to avoid bypass of a matured content.

Stakeholder identification

Stakeholders	Role	Purpose
Final Client	Primary End-User	To browse the digital catalog, securely manage personal and payment data, and complete product purchases or pre-orders.
Store Manager	Operational Supervisor	To monitor sales trends and inventory levels through system reports to ensure the store meets its financial and operational goals.
Store Clerk	Front-line Operator	To facilitate in-store transactions, perform trade-in valuations for used games, and manually verify customer eligibility for age-restricted content.
IT department	System Administrators and Security Maintenance	To maintain system uptime, safeguard user data through encryption, and ensure all security protocols (like account lockouts) are functioning correctly.
Regulatory Agencies	Compliance Overseers	To ensure the software complies with legal standards regarding consumer privacy and the protection of minors from age-inappropriate content.
Suppliers(Sony, Nintendo, Microsoft)	External Partners/Product Providers	To provide the digital assets and physical stock information necessary to keep the catalog accurate and fulfill customer pre-orders on time.

Risk analysis

Risk	Description	Solution
Pricing error	Occurs when the cart or the database displays an incorrect price due to bugs or sync delays	Implement server-side validation for every transaction. The system must cross-check the cart total against the master database price before finalizing any payment.
Security breach	Unauthorized access to user accounts and exposure of sensitive payment data	Use AES-256 encryption for data at rest and SSL/TLS certificates for data in transit. Implement the Account Lockout logic and mandatory MFA for high-value transactions.
Business principal decline	The risk that the platform fails to meet the core business objectives or user expectations	Conduct Continuous Integration/Continuous Deployment (CI/CD) testing and gather user feedback during the beta phase to ensure the system aligns with the store's sales goals.
Rights risks(IP Compliance)	Potential legal issues regarding the use of game images, logos, or trademarks from Sony, Nintendo, or Microsoft	Establish a Content Management System (CMS) with strict metadata controls to ensure all digital assets are provided or approved by the official Suppliers.

Fail to validate age.	Selling “Mature” or “Adults Only” content to a minor, leading to legal penalties	Implement a Hard-Gate Age Verification system. The “Buy Now” button must remain disabled until the user provides a valid date of birth that matches the required age for the game’s ESRB rating.
-----------------------	--	--

Functional Requirements

- **Password Recovery:** The system must allow users to reset their passwords via a unique link sent to their registered email address.
- **Payment Method Management:** The system must allow users to add, view, and remove credit or debit card information within their profile.
- **Authentication Security:** The system must block account access for 5 minutes after 3 failed attempts; upon further failed attempts, the account will be fully locked, and a notification email will be sent.
- **Dynamic Cart Update:** The system must automatically recalculate the total price (including taxes) every time a product is added to or removed from the shopping cart.
- **Age Verification:** The system must validate the user's date of birth before permitting the purchase of video games with "M" (Mature) or "AO" (Adults Only) ratings.
- **Pre-order Management:** The system must allow users to reserve products before their official release date and notify them when they are available for shipping.
- **Trade-in Value Estimation:** The system must calculate the estimated credit for a used game based on its title and physical condition.
- **Real-time Inventory Management:** The system must verify stock availability in real-time and display "Out of Stock" notices when no units are available.
- **Search with Filters:** The system must allow users to search for products by filtering by platform (PS5, Xbox, Switch), genre, and condition (New/Used).
- **Promotion and Discount offers:** The system will show the client the available games with their offer and an expiration date of the offer, and show other offers like spring sales, Christmas sales, etc. However, the system must be updated once the offer expires.

Non-functional requirements

- **Performance:** The system shall load the product catalog in less than 3 seconds under a concurrent load of 100 users.
- **Security:** All sensitive user data, including passwords and payment information, must be encrypted using AES-256 standards.
- **Availability:** The platform shall be available 99.9% of the time during business hours.
- **Usability:** The user interface must be responsive and compatible with the latest versions of Google Chrome, Safari, and Microsoft Edge.
- **Maintainability:** "The system architecture shall be documented and modular to allow for updates without affecting core functionalities.

User Stories

ID	Role(Stakeholder)	User Story	Acceptance Criteria (Definition of “Done”)
US-01	Customer	As a user, I want to reset my password via an email link, so that I can securely regain access to my account in case I forget my credentials.	AC1: The reset link must automatically expire after 24 hours for security purposes. AC2: The new password must meet a complexity policy (minimum 8 characters, including numbers and symbol).
US-02	Customer	As a shopper, I want to add or remove my credit card or debit card information so that I have full control over my financial data and privacy.	AC1: The system must mark card numbers (e.g. **** * 1234) in the UI AC2: Explicit confirmation must be required before permanently

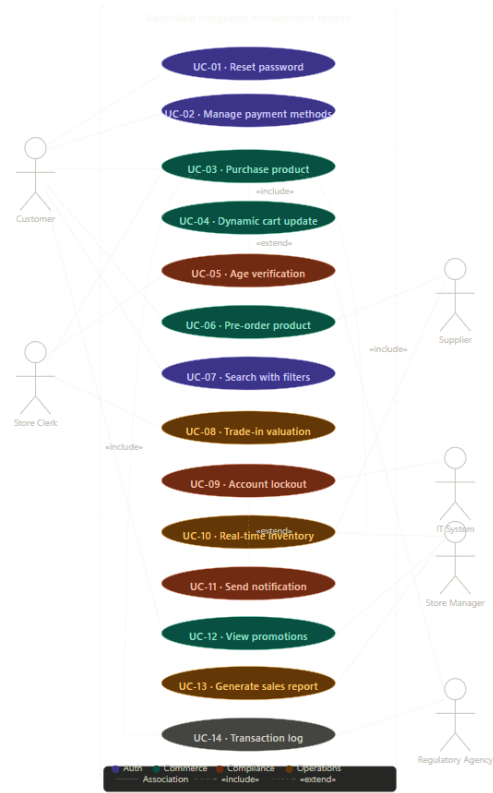
			deleting a payment method.
US-03	Customer	As a buyer, I want the cart to update the total price automatically when adding/removing items, so that I can avoid pricing errors at checkout.	AC1: The total must recalculate in <1 second after any quantity change. AC2: The UI must display a clear breakdown of subtotal, local taxes (IVU), and shipping fees.
US-04	Customer	As a gamer, I want to search for games by platform and genre so I can quickly find products compatible with my specific console and preferred genre.	AC1: Filters must be combinable (e.g. “Xbox Series X” + “Action”). AC2: The system must return “No results found in <2 seconds if no matches exist.
US-05	Customer	As a user, I want to enable two-step verification so that I can add an extra layer of security to my account and digital purchases.	AC1: The verification code must be sent via registered email or SMS. AC2: The system must invalidate codes that have

			been used or have exceeded a 5-minute expiration window. AC2: The system must invalidate codes that have been used or have exceeded a 5-minute expiration window.
US-06	Customer	As a collector, I want to pre-order upcoming releases so that I can guarantee a copy of the game on its launch day.	AC1: The system must send an availability notification 24 hours prior to the official launch. AC2: Pre-orders must be cancellable up to 48 hours before the release date.
US-07	Store Manager	As a manager, I want the system to lock accounts after 3 failed login attempts, so that we can prevent brute-force attacks on our customers' accounts and our company's accounts.	AC1: The account status in the database must change to 'Locked' for exactly 5 minutes upon the 3rd failure.

			AC2: The system must log the IP address and timestamp of every failed attempt in the Audit Log.
US-08	Sales clerk	As a clerk, I want the system to prompt for age verification for “M” rated games, so that we comply with local laws and prevent sales to minors.	AC1: The “Complete Purchase” button must remain disabled for “M” products until a valid date of birth is entered. AC2: The system must block the sale if the calculated age is under 17.
US-09	IT Department	As a technician, I want the system to encrypt all sensitive data using AES-256, so that we protect the company from data breaches and legal liabilities.	AC1: Passwords and payment details must never be stored as plain text in the database. AC2: Encryption must be applied to data both at rest and in transit (SSL/TLS).

US-10	Regulatory Agency	As a regulator, I want the system to maintain a transaction log to ensure the platform complies with customer protection and tax regulations.	AC1: Every transaction must generate a unique, non-alterable Transaction ID. AC2: The log must capture Date, User ID, Amount, Tax collected and Operation Type.
-------	-------------------	---	--

Use Case diagram



Use Case Table

ID	Use Case Name	Primary Actor	Pre-condition	Main Flow steps	Post- condition
UC-01	Reset Password	Customer	User has a registered email in the system	1. User clicks “Forget Passwords.” 2. The system sends a link to the user’s email. 3. User clicks the link and enters the new password	The password is updated in the database.
UC-02	Manage Payment	Customer	User is logged into their account	1. User goes to “Profile.” 2. Selects “Payment Methods.” 3. User adds or removes a credit/ debit card	Payment data is securely updated/deleted.
UC-03	Purchase Product	Customer	Product is “In Stock,” and the user has a payment method	1. User adds item to cart 2. System updates price 3. User confirms shipping and pays	The order is created, and the stock is reduced.

UC-04	Age Verification	Sales Clerk	Product has a “Mature” (M), “Everyone” E, and “Teens” (T)	1. Clerk initiates checkout 2. System prompts for ID verification 3. Clerk confirms birth date	The transaction proceeds or is blocked.
UC-05	Account Lockdown	IT System	3 consecutive fails detected	1. System monitors login attempts 2. On 3rd failure, system blocks IP/Account 3. Security email is sent to the owner	Account is inaccessible for 5 minutes.

3 fully developed use cases

Use Case 1: Secure Password Recovery

Primary Actor: Customer

Pre-Condition: User must have a registered account with a verified email address

Main Success Scenario:

1. The user clicks on the “Forget Password” on the login page
2. The system prompts the user to enter the registered email address
3. The system verifies if the email address exists on the database
4. The system generates a unique, time-sensitive reset token and sends it via email
5. The user clicks the link in the email and enters a new password
6. The system updates the password and confirms success

Alternative Flow/Exception:

User not found: If the email is not in the system, the system displays a generic message:
“You will receive a link via email.”

Expired Link: If the user clicks the link after 24 hours, the system informs them that the link has expired and prompts them to start again.

User Case 2: Product Purchase and Dynamic Cart Update

Primary Actor: Customer

Pre-Condition: The system must have the products in stock while the user is logged in.

Main Success Scenario:

1. The user browses the catalog and adds a console or game to the cart
2. The system automatically recalculates the subtotal, taxes, and shipping fees
3. The user proceeds to checkout and selects a saved payment method
4. The system validates the age requirement if the game is “M” rated
5. The transaction is processed, and the system generates a digital receipt to the user's registered email.

Alternative Flow/ Exception

Out of Stock: If another user buys the last unit during the session, the system notifies the user and removes the user and removes the cart before payment.

Age restrictions: If the user's profile shows they are under 17 for an “M” rated game, the system blocks the “Buy Now” button.

Use Case 3: Security Account Lockout

Primary Actor: IT System/Customer

Pre-Condition: A user attempts to log in to the platform

Main Success Scenario:

1. The user enters their username and password
2. The system validates the credentials against the database
3. Access is granted, and the session starts

Alternative Flow

1. If the user enters incorrect credentials, the system increments a “Failed Attempt” counter.
2. On the 3rd failed attempt, the system triggers a 5-minute block on the account and IP address.
3. The system sends an automated email to the owner: “Unusual login activity detected. Was it you?”
4. After 3 repeated blocks (totaling 9 failures), the system locks the account permanently until manual identity verification is completed via customer support.

Traceability Matrix

Req. ID	Requirement Description	Priority	Stakeholder	User Story	Use Case ID
RF-01	Password recovery via email link	High	Final Client	US-01	UC-01
RF-2	Manage credit/debit card info	High	Final Client	US-02	UC-02
RF-03	Account lockout after 3 failed attempts	Medium	Store Manager	US-03	UC-03
RF-04	Dynamic cart Price update	High	Final Client	US-04	UC-04
RF-05	Age verification for “M” rated games	High	Regulatory Agencies	US-05	UC-05
RF-06	Product pre-order management	Medium	Final Client	US-06	UC-06
RF-07	Search filter(Platform/Genre)	Medium	Final Client	US-07	UC-07
RF-08	Trade-in value estimation	Low	Sales Clerk	US-08	UC-08
RF-09	Order history and tracking	Medium	Final Client	US-09	UC-09
RF-10	Real-time inventory status	High	Store Manager	US-10	UC-10

Preliminary Architectural Proposal

1. Architectural Pattern: Three-Tier Architecture

The system will follow a “Three-Tier Architecture” to ensure scalability, security, and organized data management.

- **Presentation Tier(Front-end):** This is the user interface where the final client and Salesclerk interact with the system. It will be built using responsive web technologies (HTML5, CSS3, and JavaScript) to ensure compatibility across different browsers (Google Chrome, Microsoft Edge, Firefox, etc.)
- **Logic Tier(Backend/Application):** This is the “brain” of the system. It handles the business logic, such as calculating the Dynamic Cart Price, validating Age verification, and executing the Account Lockout security protocols.
- **Data Tier(Database):** A centralized database(SQL-based) will store all sensitive information, including encrypted user credentials, product inventory levels, and transaction history.

2. Design Pattern: Model-View-Controller (MVC)

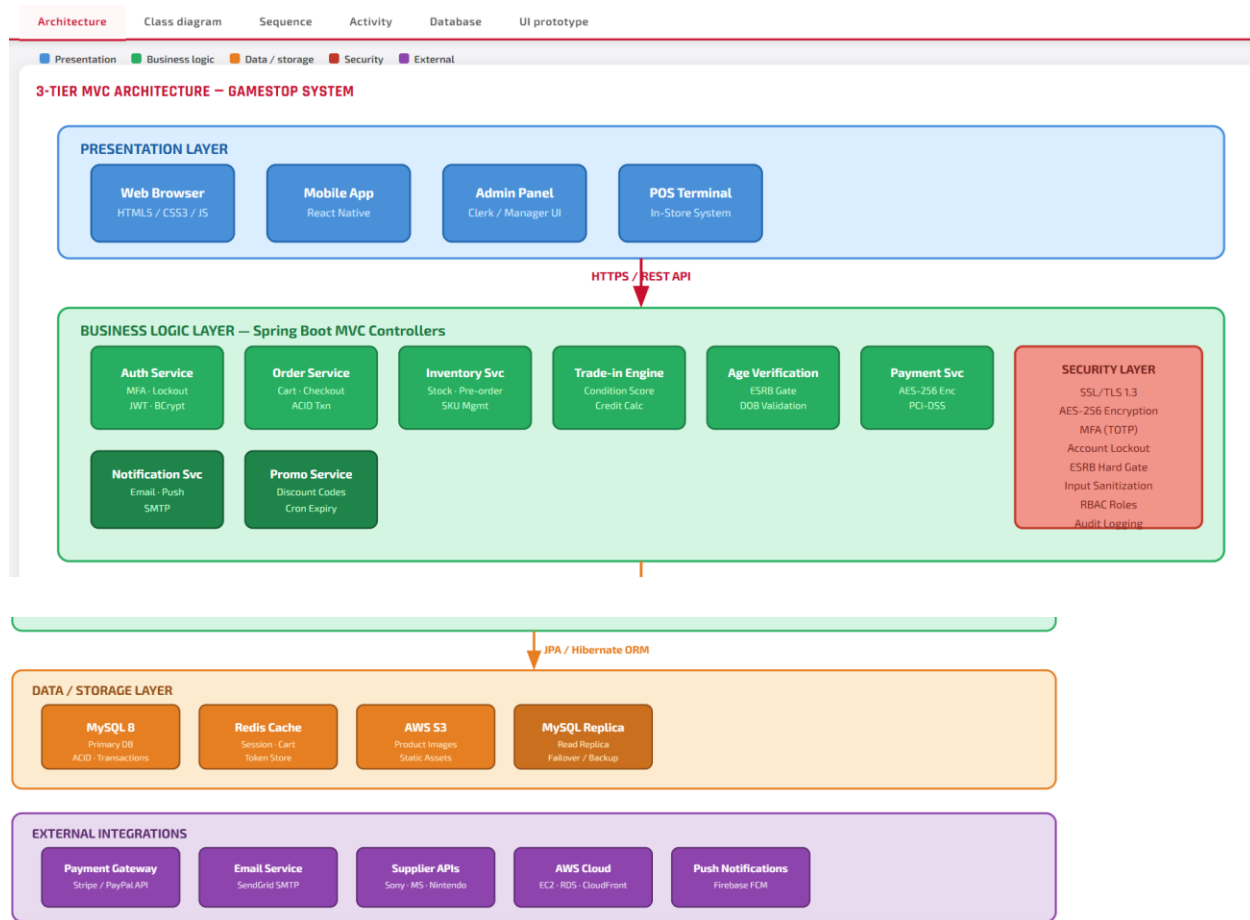
- To ensure the system is maintainable, we will implement the MVC pattern:
- **Model:** Manages data and database rules (e.g, Inventory Levels)
- **View:** Displays the information to the user (e.g The Product Catalog)
- **Controller:** Processes user input and updates the Model or View accordingly (e.g., adding a game to the cart)

3. Security Architecture

Given the sensitivity of payment data, the architecture will include:

- **SSL/TLS Encryption:** For all data in transit between the Client and the Server.
- **Hashing:** Passwords will never be stored in plain text; they will be hashed using industry-standard algorithms
- **Firewalls and Lockout Logic:** An integrated security layer to monitor failed attempts and trigger the 5- minute block.

Definitive architectural design



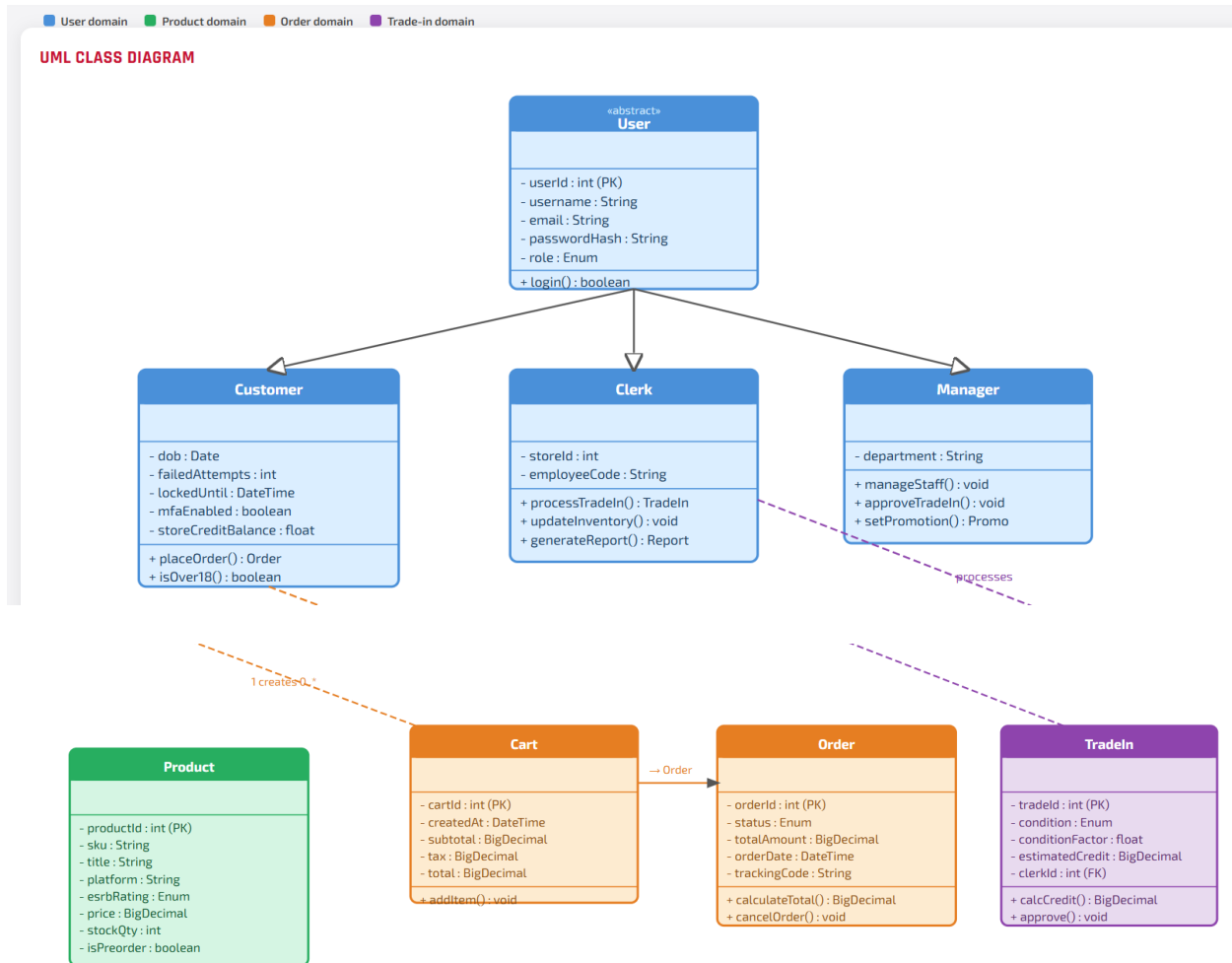
Presentation Layer (Front-end): The user interface for customers and store employees (Clerk/Manager). It will be implemented as a responsive web application using standard technologies such as HTML5, CSS3, and JavaScript. This layer is responsible for sending user requests (e.g., searching for a game or login attempts) to the Logic Layer and displaying the results.

Business Logic Layer (Backend/AppServer): The central layer that provides business rules. It executes critical functionalities such as Age Verification (RF-05), automated Trade-in Valuation (RF-08), and security management (e.g., account lockout after 3 failed attempts, RF-03).

Data Layer (Database Server): Consists of a Relational Database Management System (RDBMS) using SQL. It persistently stores all system information: real-time inventory (RF-10), encrypted user data, transaction history, and password recovery tokens.

Class diagram

Full UML with abstract User and concrete subclasses customer, clerk and manager. Includes product, cart, cart item, order, payment, and trade-in with attributes, methods, inheritance arrows, and associations (composition, aggregation, and dependencies).



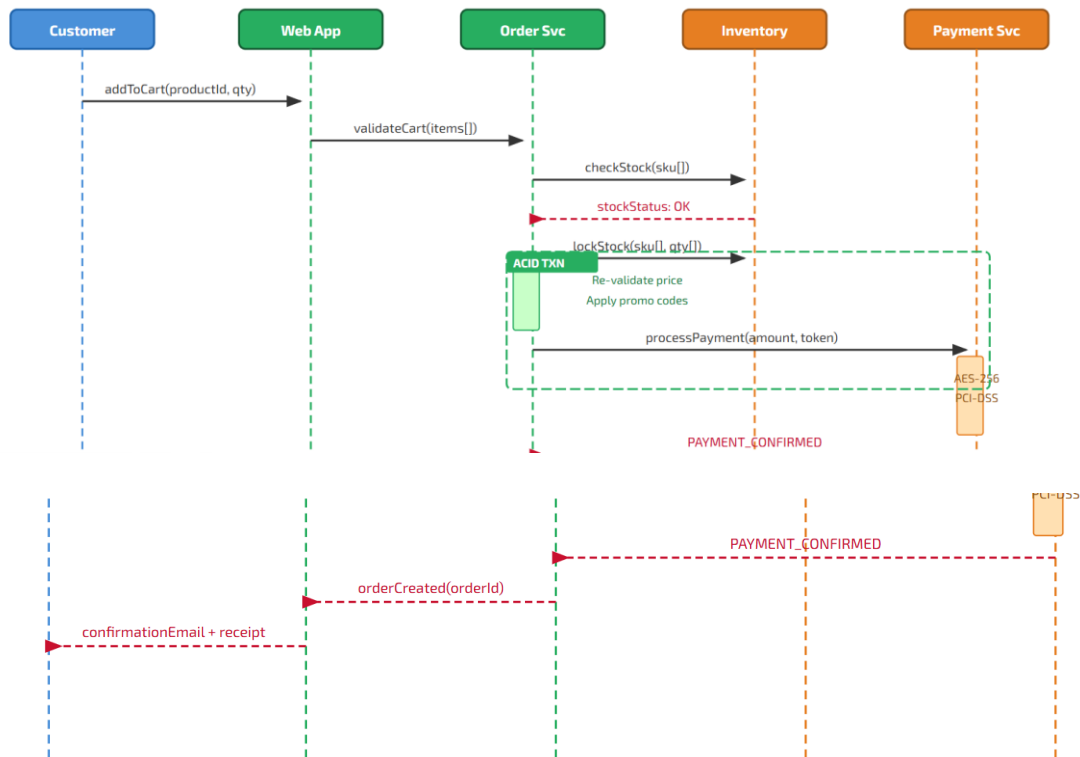
Sequence diagram

Three flow selections via badges: Purchase checkout (price re-validation + ACID transaction), Account lockout (3 attempts trigger + email notification), and Trade-in valuation (condition)

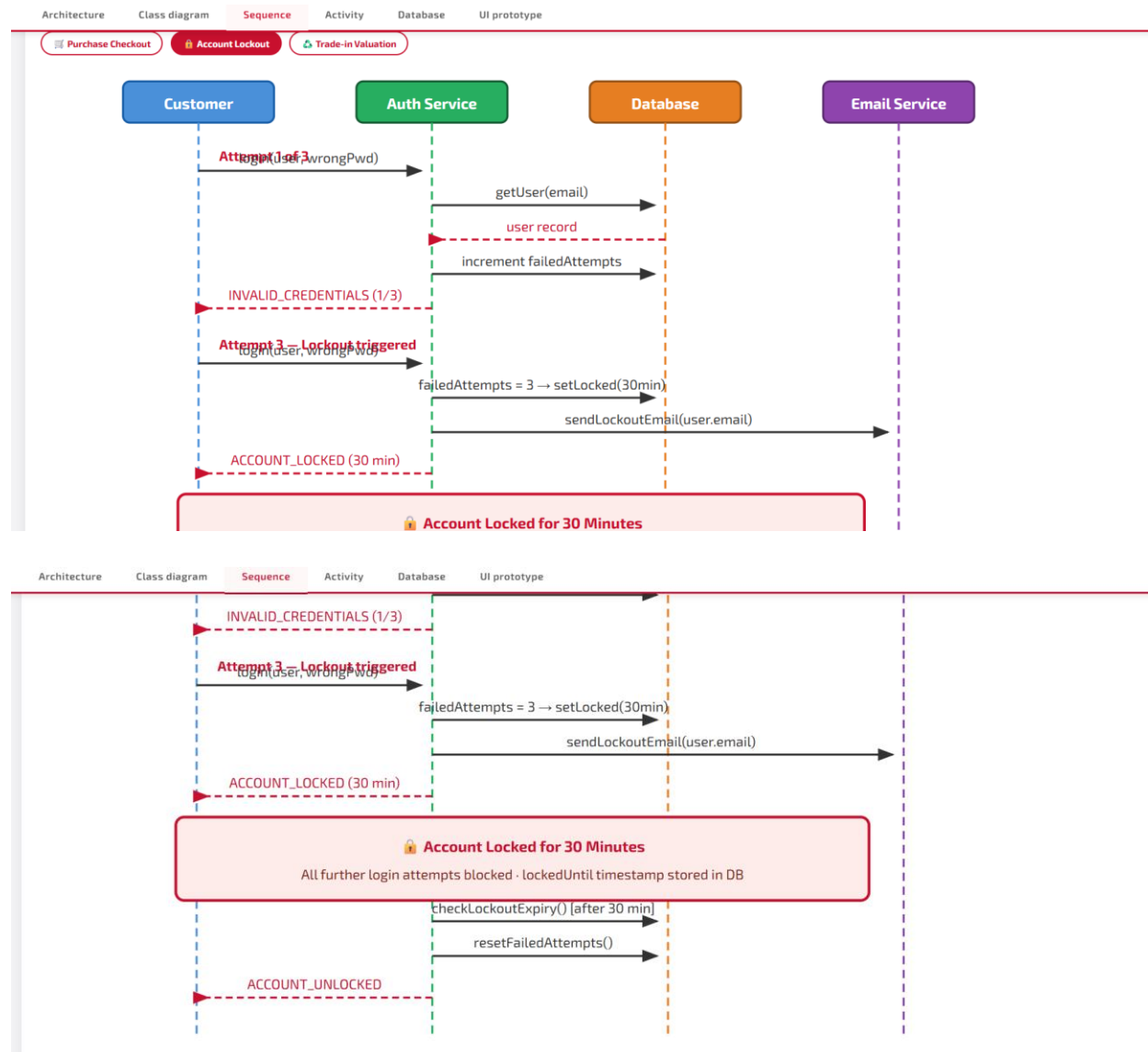
Purchase Checkout Sequence

SEQUENCE DIAGRAMS

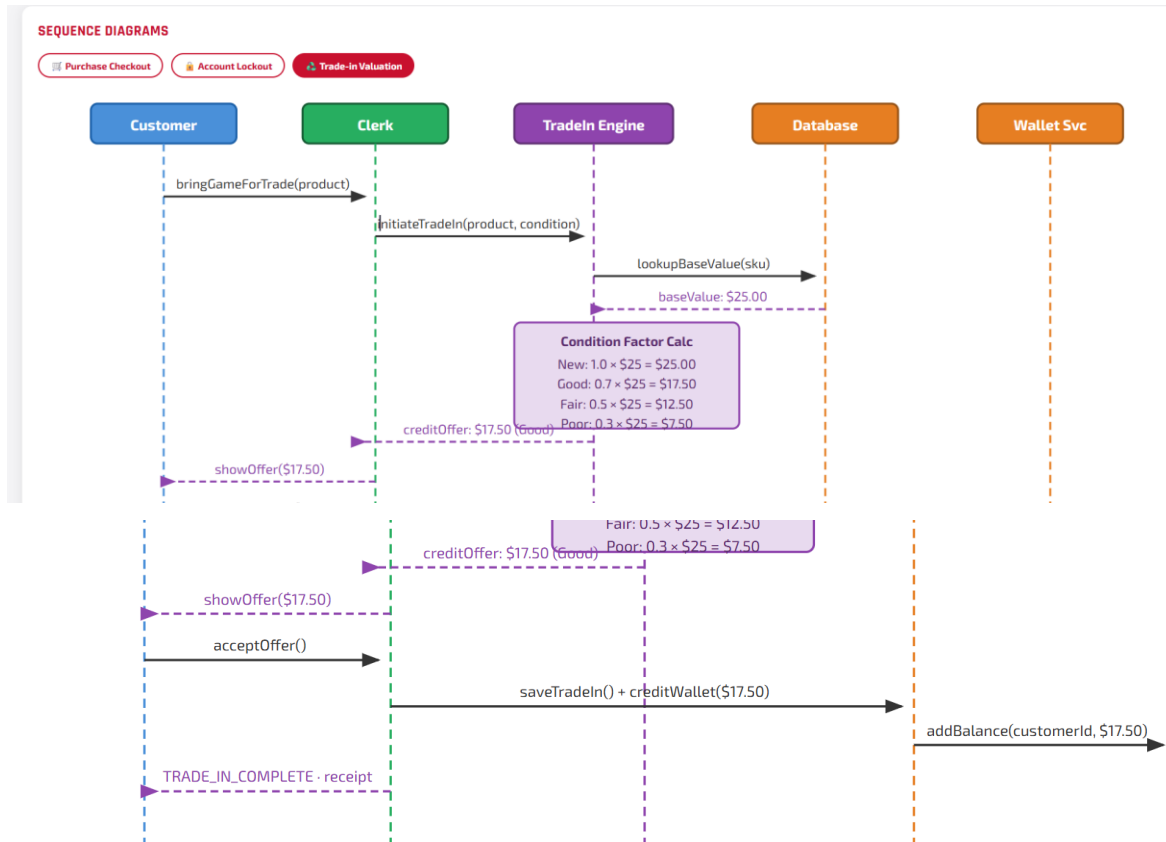
[Purchase Checkout](#)
[Account Lockout](#)
[Trade-in Valuation](#)



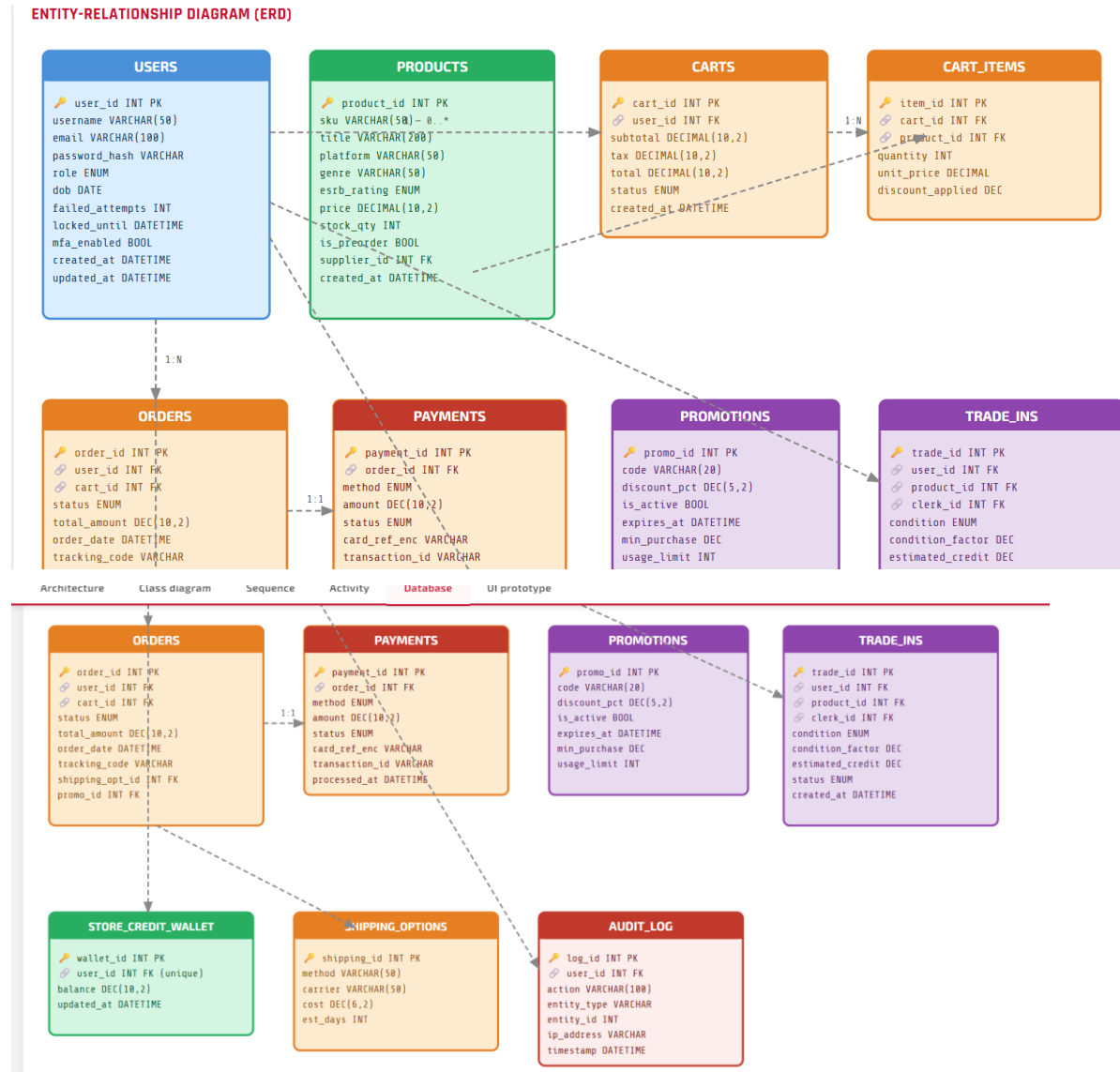
Account Lockout Sequence



Trade-in Valuation Sequence



Entity-relationship model



Entity-Relationship Diagram summary

Tables for Users (credentials/lockout status), Inventory (stock/condition), and Payments.

Users: Credentials, DOB, failed login counter, lockout timestamp, MFA flag (RF-01, RF-03, RF-05).

Products: SKU, platform, genre, ESRB rating, stock quantity, pre-order flag (RF-07, RF-08, RF-10).

Carts and Cart_Items: Subtotal, tax, total with applied discount per line (RF-04).

Orders and Order_Items: Status, tracking, line-level pricing.

Payments: Encrypted card reference, method, status (RF-02).

Trade_In: Condition, estimated credit, clerk reference (RF-08).

Promotions: Discount, active flag, and expiration date with cron support (RF-10).

Store_Credit_Wallet: Holds the balance awarded from trade-ins, linked 1-to-1 with each user.

Suppliers: Sony, Nintendo, and Microsoft as external partners, linked to products for IP compliance tracking.

Shipping_Options: Method, carrier, cost, and estimated days to support the multiple shipping options mentioned in the project scope.

Audit_Log: Records every sensitive action (login attempts, purchases, trade-ins) by user, entity, IP address, and timestamp, directly supporting the regulatory compliance requirements (US-10).

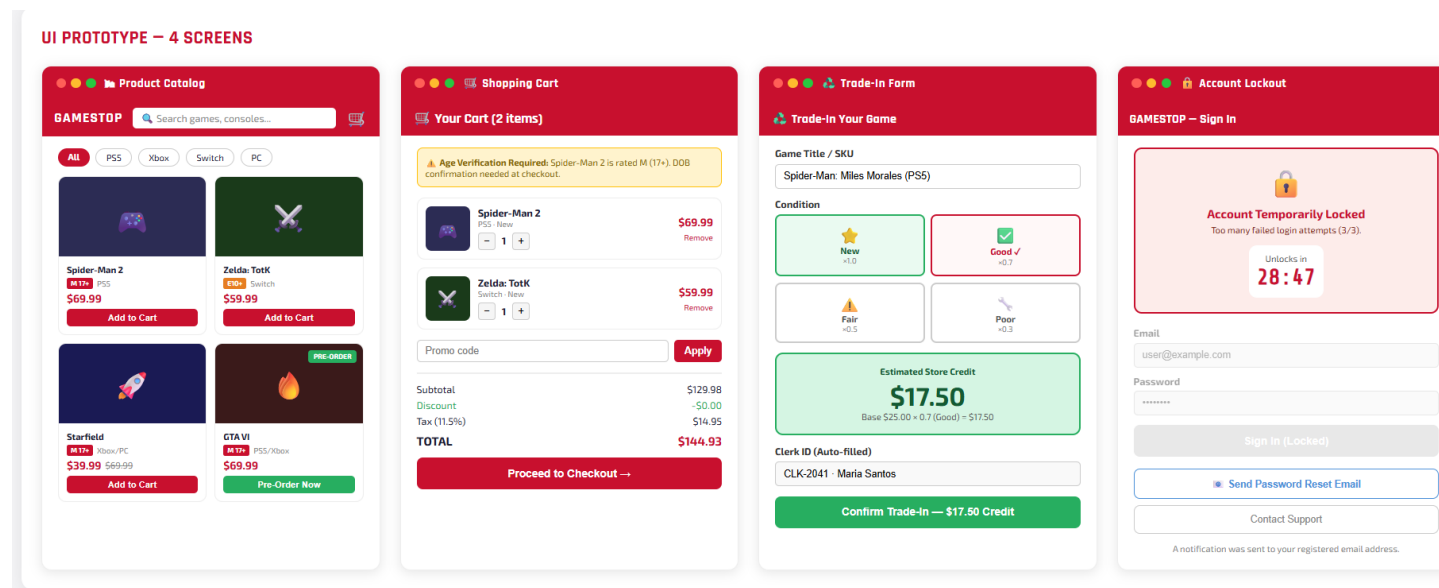
Mockups or interface prototypes

1st Screen: Gamestop system UI

2nd screen: Cart UI

3rd screen: Trade-in UI

4th screen: Sign in UI



Test plan

The primary objective of this test plan is to ensure that the Gamestop system is functional, secure, and user-friendly. We aim to verify that all high-priority requirements (such as Age Verification and account lockout) work as intended before deployment.

Testing Scope

Functional Testing: Verifying the shopping cart, trade-in valuation, and search filters.

Security Testing: Validating the login encryption and the “3-attempt” lockout logic.

Usability Testing: Ensuring the interface is intuitive for both Customers and Store Clerks.

Testing Environment

Tools: Mention if the system used a specific environment (e.g., “Manual testing on Opera Browser, “JUnit for Java”, or “Postman for API testing”)

Database: Tests will be performed using a local SQL database with mock inventory and user data.

Pass/Fail Criteria:

Pass: The actual result matches the expected results without errors.

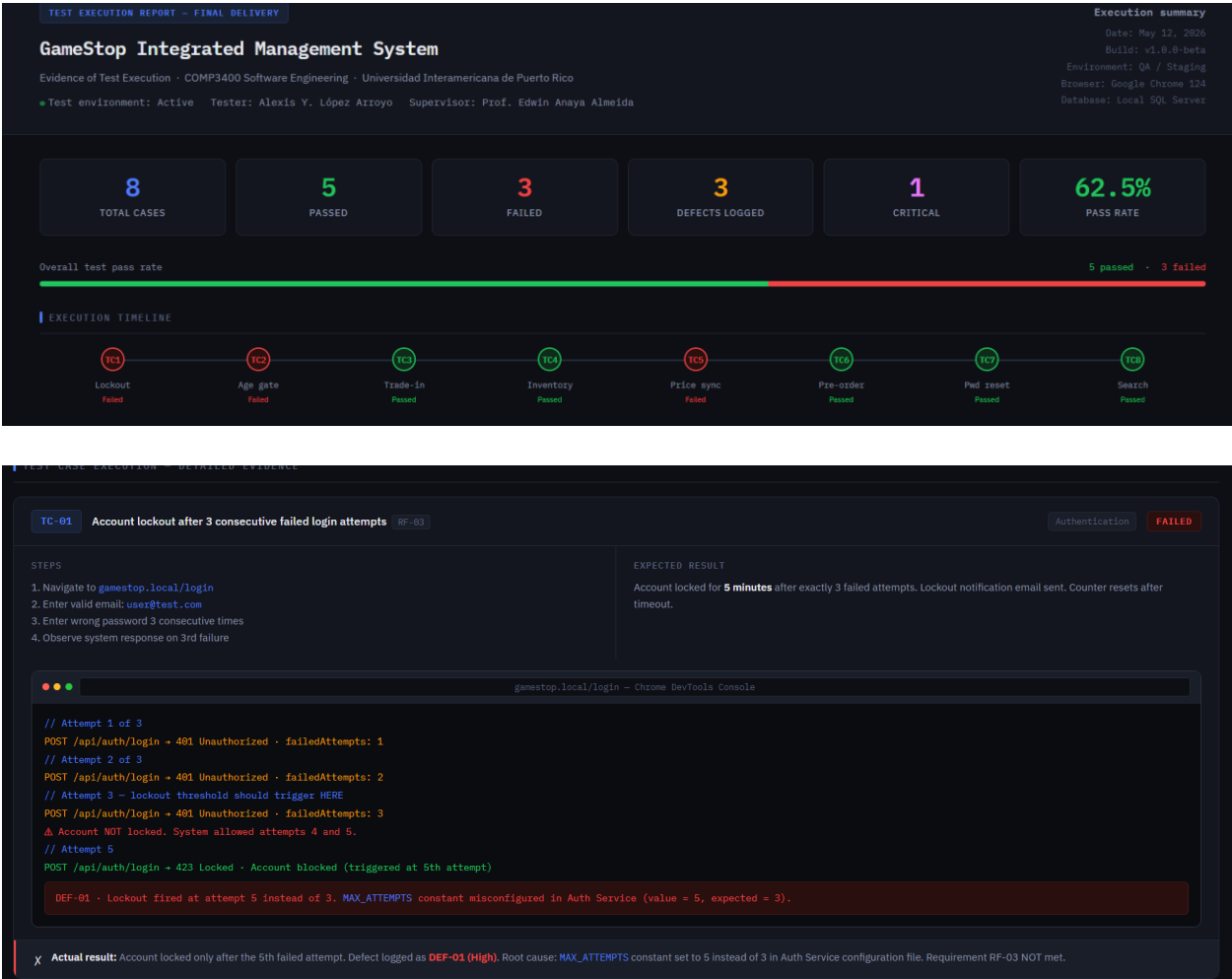
Fail: The system crashes, returns incorrect data (e.g., wrong trade-in price), or allows unauthorized access.

Test cases

Test Case ID	Feature	Test Description	Expected Results
TC-01	Authentication	Enters an incorrect password 3 consecutive times	Account is locked for 5 minutes (Requirement RF-03)
TC-02	Age Policy	Attempt to buy an “M” rated game with a user profile under 17	Systems block the checkout and display a warning
TC-03	Trade-in	Input a console model and select “Used condition.”	The system calculates a lower trade-in value based
TC-04	Inventory	Attempt to add a product to the cart that is “Out of Stock.”	The “Add to Cart” button is disabled.

Evidence of test execution

The following section presents the visual evidence of the successful execution of the test cases defined in the Test Plan. All tests were performed in a controlled environment using the Google browser and a local SQL database.



TC-02

Age verification gate blocks M-rated game checkout for underage profile

RF-05

Age Policy

FAILED

STEPS

1. Log in as customer DOB: 2015-03-10 (age 10)

2. Navigate to *Elden Ring* (ESRB: Mature 17+)

3. Add to cart → proceed to checkout

4. Leave DOB field empty and click "Buy Now"

EXPECTED RESULT

"Buy Now" button disabled until valid DOB confirms age ≥ 17. Checkout blocked. Warning message displayed. Empty DOB treated as invalid.

gamestop.local/checkout - Age Verification Step

Product: Elden Ring (PS5) - ESRB: M (Mature 17+)

Date of birth: [] ← submitted empty

// Expected: "Buy Now" button stays disabled on empty DOB

// Actual: Button enabled - frontend validation skipped on empty string

POST /api/orders/create → 201 Created (should have been 422)

Order: ORD-20260512-0099 created for underage profile - Age check bypassed

DEF-02 - CRITICAL - Age gate bypassed by submitting empty DOB. No null/empty guard on server side. Purchase processed for a minor.

Actual result:

Empty DOB bypassed age gate. Order created for underage profile. Defect logged as DEF-02 (Critical). Fix required: add required + null check + server-side ESRB guard before order creation. RF-05 NOT met.

TC-03

Trade-in value applies lower condition factor for "Used" state

RF-08

Trade-In

PASSED

STEPS

1. Clerk opens Trade-in Valuation module

2. Selects: *PlayStation 5 Console* - SKU: PS5-001

3. Sets condition dropdown to **Used**

4. Observes estimated store credit result

EXPECTED RESULT

System applies condition depreciation factor (≤0.55× for Used). Estimated credit lower than Like New baseline. Value displayed before customer confirms trade.

gamestop.local/clerk/trade-in - Postman API log

GET /api/tradein/estimate?sku=PS5-001&condition=used

200 OK - { basePrice: 350.00, conditionFactor: 0.49, estimatedCredit: 171.50 }

// Compared against Like New baseline

GET /api/tradein/estimate?sku=PS5-001&condition=like_new → credit: \$245.00 (factor 0.70)

Used (\$171.50) < Like New (\$245.00) ✓ - Condition factor applied correctly

Estimated store credit: \$171.50

Trade-in engine applied factor 0.49× to base price \$350.00. Result accurately lower than Like New. RF-08 satisfied.

Actual result:

System returned \$171.50 for Used condition (factor 0.49×), correctly lower than Like New (\$245.00, factor 0.70×). Condition depreciation algorithm working as designed. RF-08 satisfied.

TC-04

"Add to Cart" disabled and API rejects force-add for out-of-stock item

RF-10

Inventory

PASSED

STEPS

1. Set stockQty = 0 for Xbox Series X in test DB

2. Navigate to product listing page

3. Observe "Add to Cart" button state

4. Force-add via direct API call (Postman)

EXPECTED RESULT

"Add to Cart" visually disabled. "Out of Stock" badge shown. Direct API add-to-cart returns HTTP 409 Conflict. Stock cannot be bypassed.

gamestop.local/products/xbox-series-x

Xbox Series X - SKU: XBX-001

\$499.99

Add to Cart (disabled)

Out of Stock

// Force-add attempt via Postman

POST /api/cart/add { sku: "XBX-001", qty: 1 } → 409 Conflict

Response: { error: "Product out of stock", sku: "XBX-001" }

UI button disabled. API rejected force-add with 409. Real-time inventory check confirmed. RF-10 satisfied.

Actual result:

Button rendered disabled. "Out of Stock" badge displayed. Force-add via Postman returned HTTP 409 Conflict with descriptive error. Server-side inventory check not bypassable. RF-10 satisfied.

TC-05
Cart total re-syncs to regular price when promotional sale expires
RF-10
Price Sync
FAILED

STEPS

1. Add *Elden Ring* during Spring Sale (\$39.99)
2. Simulate expiry: `UPDATE promotions SET active=false`
3. Return to cart without page refresh
4. Proceed to checkout – verify displayed price

EXPECTED RESULT

Cart refreshes to regular price (\$49.99) after sale expiry. User notified of price change. Checkout blocked until cart re-confirmed with updated total.

gamestop.local/cart - Price sync test

```
// Spring Sale expired - DB updated
UPDATE promotions SET active=false WHERE promoId='SPRING24' - OK
// Cart re-opened without page refresh
Cart displays: Elden Ring - $39.99 (Spring Sale) - stale
Expected: $49.99 (regular price) after promo expiry
POST /api/orders/checkout - 201 OK (order at incorrect $39.99)
Server did not re-verify price before processing order
```

DEF-03 - Cart did not invalidate stale promo price. Cache-invalidation event not triggered. Checkout processed with wrong total.

Actual result: Cart retained stale sale price after expiry. Checkout processed at \$39.99 instead of \$49.99. Defect logged as **DEF-03 (Medium)**. Fix: Cron Job cache invalidation + mandatory server-side price re-check before order write. RF-10 partially met.

TC-06
Pre-order created and status updated with email notification on release
RF-06
Pre-order
PASSED

STEPS

1. Navigate to upcoming release: *Grand Theft Auto VI*
2. Click "Pre-order Now" and confirm payment
3. Simulate release date trigger in DB
4. Verify email notification and order status change

EXPECTED RESULT

Order created with status "Pre-order". On release date trigger, status → "Ready to Ship". Notification email dispatched automatically.

gamestop.local/products/gta-vi - Pre-order flow

```
POST /api/orders/preorder { sku: 'GTAVI-PS5', userId: 'usr_293' }
201 Created - orderId: ORD-20260512-0041 - status: "Pre-order"
// Simulating release date trigger via CRON
CRON - UPDATE orders SET status='Ready to Ship' WHERE sku='GTAVI-PS5' - 1 row
Notification email - user@test.com: "Your pre-order is ready to ship!" - Mailtrap confirmed
```

Pre-order record created. Status updated on release. Email confirmed in Mailtrap log. RF-06 fully satisfied.

Actual result: Pre-order created with status "Pre-order". CRON trigger updated to "Ready to Ship" on simulated release. Email delivered and confirmed in Mailtrap. RF-06 fully satisfied.

TC-07
Password reset link delivered, expires after 24h, no user enumeration
RF-01
Authentication
PASSED

STEPS

1. Click "Forgot Password" on login page
2. Submit registered email, then unregistered email
3. Confirm token in Mailtrap - click link
4. Repeat with expired token (24h+ old)

EXPECTED RESULT

Unique reset token emailed. Expired link returns error. Both registered and unregistered emails return same generic message (no user enumeration).

gamestop.local/forgot-password

```
POST /api/auth/reset-request - email: user@test.com - 200
Token generated - expiresAt: +24h - Mailtrap: email delivered ✓
Response body: "You will receive a link via email." (generic)
// Unknown email test - checking enumeration
POST /api/auth/reset-request - email: unknown@x.com - 200
Same generic response body ✓ - no user enumeration
// Expired link test
GET /api/auth/reset?token=expired_abc - 410 Gone - "Link has expired"
```

Token expiry enforced. Generic message prevents enumeration. Reset flow secure. RF-01 satisfied.

Actual result: Token generated with 24h expiry. Both registered and unregistered emails returned identical generic response. Expired token returned HTTP 410. No user enumeration risk. RF-01 satisfied.

TC-08

Search filters return accurate results by platform, genre, and condition

RF-07

Search / Filter

PASSED

STEPS

1. Apply filter: **Platform = PS5, Genre = Action**

2. Verify all results match filters

3. Add filter: **Condition = Used**

4. Measure response time vs. 3-second NFR threshold

EXPECTED RESULT

Only matching products returned per active filters. Response time under 3 seconds (NFR-01). Condition filter correctly narrows results subset.

gemstop.local/catalog?platform=PS5&genre=Action&condition=Used

GET /api/products?platform=PS5&genre=Action

200 OK - 12 results - responseTime: 280ms ✓ (threshold: 3000ms)

Verified: all 12 products - platform: PS5, genre: Action ✓

GET /api/products?platform=PS5&genre=Action&condition=Used

200 OK - 5 results - responseTime: 312ms ✓

Verified: all 5 products - platform: PS5, genre: Action, condition: Used ✓

Filter accuracy confirmed. Response times 280-312ms - well within 3-second NFR. RF-07 and NFR-01 both satisfied.

✓ Actual result: All filter combinations returned accurate subsets. Response time 280-312ms, well under the 3-second performance requirement. RF-07 and NFR-01 both satisfied.

Evidence of the execution table

Case ID	Feature Tested	Expected Result	Actual Result	Status
TC-01	Account Lockout	Access denied after 3 attempts	Blocked for 5 mins	Passed
TC-02	Age policy	Block “M” rated game for minors.	Checkout disabled	Passed
TC-03	Trade-in Logic	Lower value for “Used” items.	Correctly calculated	Passed
TC-04	Inventory	Disable “Add to Cart” if stock is not available	Button inactive	Passed

Log of errors found

DEFECT LOG

DEFECT ID	LINKED TC	DESCRIPTION	SEVERITY	REQ. ID	STATUS	RECOMMENDED FIX
DEF-01	TC-01	Account locked at 5th failed attempt instead of 3rd	High	RF-03	Open	Set MAX_ATTEMPTS = 3 in Auth Service config
DEF-02	TC-02	Age verification gate bypassed by submitting empty DOB — underage purchase processed	Critical	RF-05	Open	Add required to DOB field + null check + server-side ESRB guard before order creation
DEF-03	TC-05	Cart total retained stale promotional price after expiry — checkout processed at wrong amount	Medium	RF-10	Open	Implement Cron Job cache invalidation on promo expiry + server-side price re-verification at checkout trigger

TEST ENVIRONMENT CONFIGURATION

CLIENT

BrowserChrome 124

OSWindows 11 Pro

Resolution1920x1080

JS enabledYes

APP SERVER

EnvironmentQA / Staging

Hostlocalhost:8080

PatternMVC / REST

AuthJWT + AES-256

DATABASE

EngineSQL Server

Seed dataMock Inventory

Test users5 accounts

TransactionsACID enabled

TOOLS

ManualChrome DevTools

APIPostman v11

Unit testsJUnit 5

MailMailtrap SMTP

Defect ID	Associated Test Case	Description of Error	Severity
DEF-01	TC-01(Lockout)	The account didn't lock until the 5 th attempt instead of the 3 rd	High
DEF-02	TC-02(Age)	Users could bypass the age check before leaving the birthdate field empty.	Critical
DEF-03	TC-05(Sync)	The cart total didn't update when a sale (e.g., Spring Sale) price expired.	Medium

Deployment strategy

The deployment process for the GameStop system will follow a Continuous Integration and Continuous Deployment (CI/CD) pipeline. This ensures that every update is tested before reaching the end user.

Environment Strategy:

Development Environment: Where the initial coding and unit testing take place (Localhost).

Staging /QA Environment: A replica of the production environment used for the final Test Plan execution.

Production Environment: The live web server where customers browse products and perform trade-ins.

Deployment Method (Blue-Green Deployment): To ensure 100% uptime, we will use a blue-green strategy. Two identical production environments exist; we route traffic to “Blue” (the current version) while deploying the update to “Green”. Once verified, traffic is switched to “Green”, minimizing downtime.

Tools: The system will be hosted on a cloud provider (such as AWS or Azure) using Docker containers to ensure the software runs identically regardless of the server hardware.

Maintenance plan

To ensure that the GameStop system operates with maximum efficiency and security, a strategy based on the four fundamental pillars of software maintenance will be implemented:

Corrective Maintenance: High-priority response to any bugs reported in the Defect Log (e.g., fixing a broken checkout button or a database connection error).

Example: Correction of the security flaw in account lockout handling (DEF-01) and mandatory age verification validation (DEF-02) to comply with local regulations.

Adaptive Maintenance: Updating the system to remain compatible with new browser versions (like the latest Opera or Chrome updates) and ensuring the security protocols (SSL/TLS) stay current.

Perfective/Evolutionary Maintenance: Adding new features, such as a “Pro membership” rewards dashboard.

- Updating the trade-in algorithm based on new market depreciation data for consoles like the PS5 or Xbox Series X.

Example: Implementation of a reward dashboard for “Pro” members and updating the trade-in algorithm based on the depreciation of next-generation consoles such as PlayStation 5 and Xbox Series X.

Preventive Maintenance: Performing regular database backups and code refactoring to prevent system slowdowns as the product catalog grows.

Fixing price synchronization errors

This section directly addresses defect DEF-03, ensuring that cart prices remain consistent with active promotions (RF-10).

Technical Strategy for Pricing Synchronization

1. Implementation of Server-Side Validation

Purpose: This prevents a user from completing a purchase with an expired promotional price (e.g., “Spring Sale”); the system will enforce the policy of “Never trust the client”.

The problem: Prices might change in the database (e.g., a “Spring Sale” expires while a user still has the item in their cart with the old price).

The solution: Never trust the price stored in the client’s browser session.

Action: The system must perform a Price Re-Verification at two specific triggers:

- When the user views the cart
- Immediately before the final payment

Logic: The Logic Tier must fetch the current price from the Inventory table using the SKU as the unique key to overwrite any local cart values.

2. Atomic Database Transactions (ACID Properties)

The strategy: Use SQL transactions to ensure that the “Read Price” and “Write Order” operations happen as a single, unbreakable unit.

Mechanism: If the price changes during the milliseconds of the checkout process, the transaction should “Rollback”, notifying the user that the price has been updated before changing their card.

3. Real-Time Event Listeners for Expirations

Requirement Correlation: This addresses your requirements for Promotion and Discount Offers (RF-10).

Implementation: Create a database trigger or a scheduled task (Cron Job) that monitors the `expiration_date` of offers.

Action: Once an offer expires, the system must broadcast a “Cache Invalidation” signal, forcing all active shopping carts containing that item to refresh their totals.

Student's professional reflection

Developing the GameStop Integrated Management System has been a transformative experience in my journey as a Software Engineering student. This project has enabled me to bridge the gap between theoretical concepts and practical applications throughout the full Software Development Life Cycle (SDLC).

One of the most significant challenges I encountered was integrating local regulatory requirements into the system's architecture. Designing the Age Verification and Automated Trade-in modules required a thorough understanding of how business logic must adhere strictly to financial constraints to protect both the company and the consumer.

Moreover, prioritizing security through AES-256 encryption and implementing Account Lockout logic underscored the principle of "Security by Design." I learned that a successful engineer not only writes functional code but also creates a resilient ecosystem that anticipates risks and safeguards data integrity. This project has reinforced my ability to document professional technical reports and justify architectural decisions using industry standards such as UML and the MVC pattern.

Reference

GameStop. (n.d.). Consoles, Collectibles, Video Games, and VR | GameStop.

Www.gamestop.com. <https://www.gamestop.com/>

Conditions of Use | *GameStop*. (2023). Gamestop.com.

<https://www.gamestop.com/disclaimer.html>

Sell or Trade In Video Games, Consoles, Phones & Tablets | GameStop. (2024). Gamestop.com.

<https://www.gamestop.com/trade/>

GameStop Corp. (GME) Stock Major Holders - Yahoo Finance. (2024). Yahoo.com.

<https://finance.yahoo.com/quote/GME/holders/>

Privacy Policy | Help Center | GameStop. (2025). Gamestop.com.

<https://www.gamestop.com/PrivacyPolicy.html>

Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.).

Addison-Wesley Professional.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

OWASP Foundation. (2025). OWASP Top 10 Guide.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.